Ejemplo de aplicación Reducción

Universidad Nacional de La Plata



Facultad de Informática

Prof. Dr. Adrian Pousa

Agenda

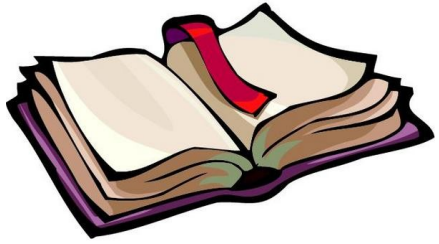
I. Reducción

II. Suma por reducción

   i. Estrategia

   ii. Alternativa para mejora de acceso a memoria

   iii. Código CUDA



Agenda

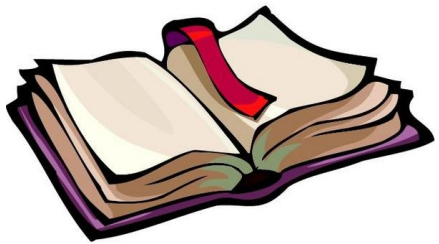
I. Reducción

II. Suma por reducción

   i. Estrategia

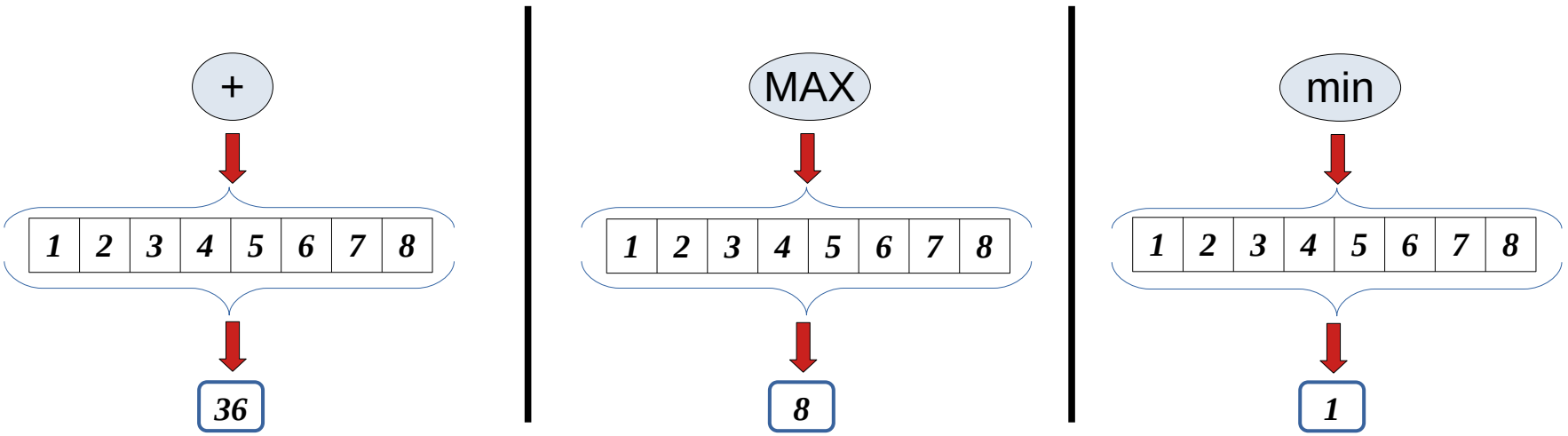
   ii. Alternativa para mejora de acceso a memoria

   iii. Código CUDA



Reducción

- **Reducción:** reducir los valores de un vector a un valor simple.
- Es posible hacerlo aplicando un operador asociativo:
  - +, *, MAX, min, etc.



Agenda

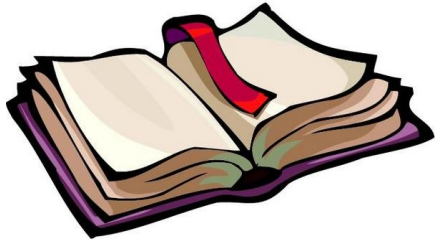
I. Reducción

II. Suma por reducción

   i. Estrategia

   ii. Alternativa para mejora de acceso a memoria

   iii. Código CUDA



Agenda

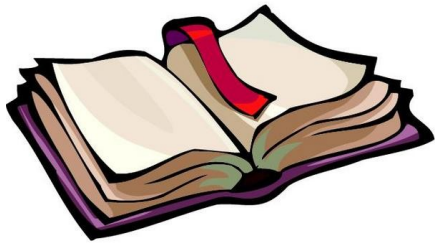
I. Reducción

II. Suma por reducción

   i. Estrategia

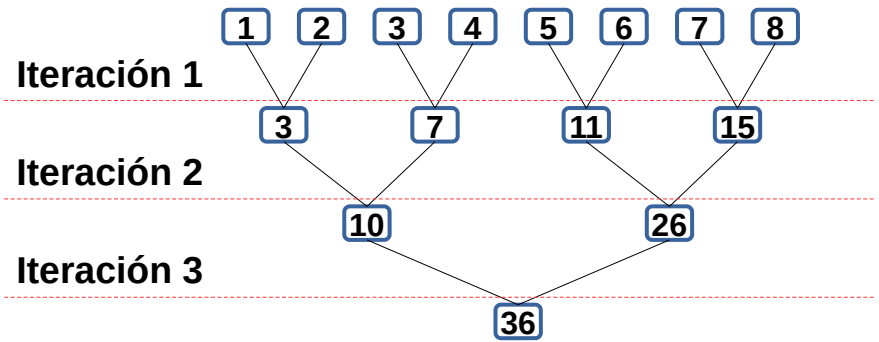
   ii. Alternativa para mejora de acceso a memoria

   iii. Código CUDA



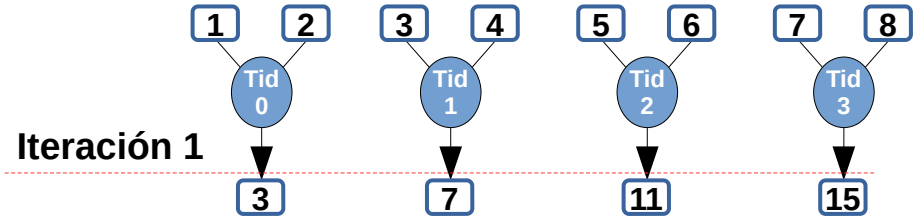
Suma por reducción
Estrategia

- Dado un vector de N elementos, la solución de una **suma por reducción** en GPU consiste en una implementación basada en árbol.
- Se requieren varias iteraciones, en cada iteración se realiza una nueva llamada al kernel y se aprovecha el hecho de que los valores de las variables en memoria global no cambian entre llamados al kernel.

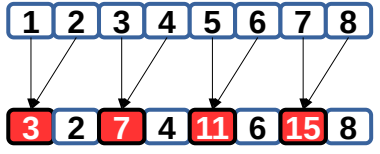


Suma por reducción
Estrategia

- Para la primera iteración se crean $N/2$ hilos.
- Cada hilo suma su posición y la siguiente.

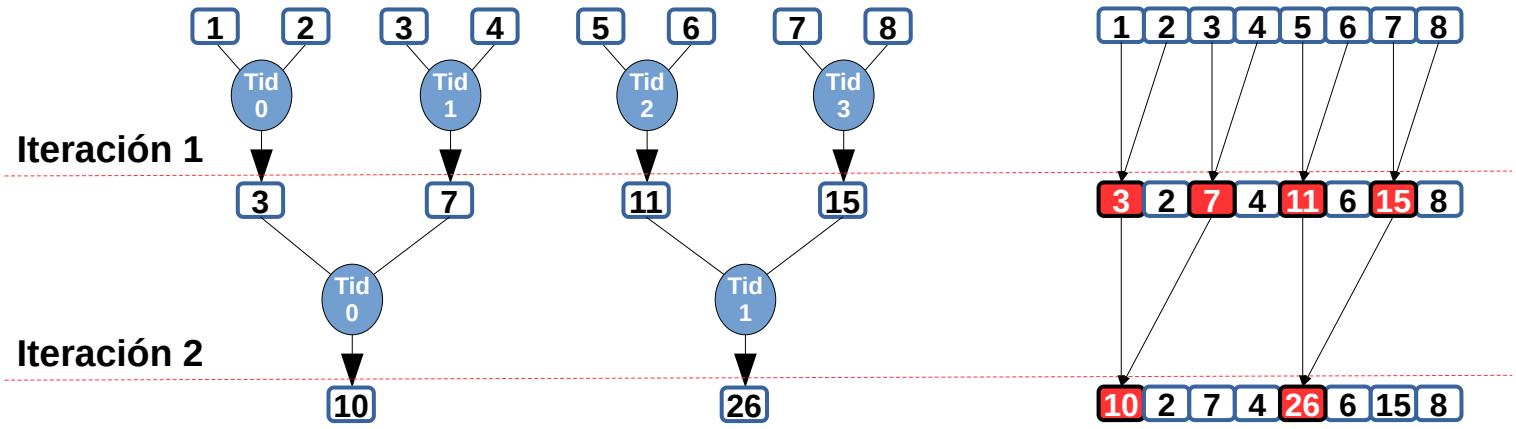


- Para minimizar el espacio de almacenamiento, el resultado de cada suma parcial se deja en la posición del primer operando.



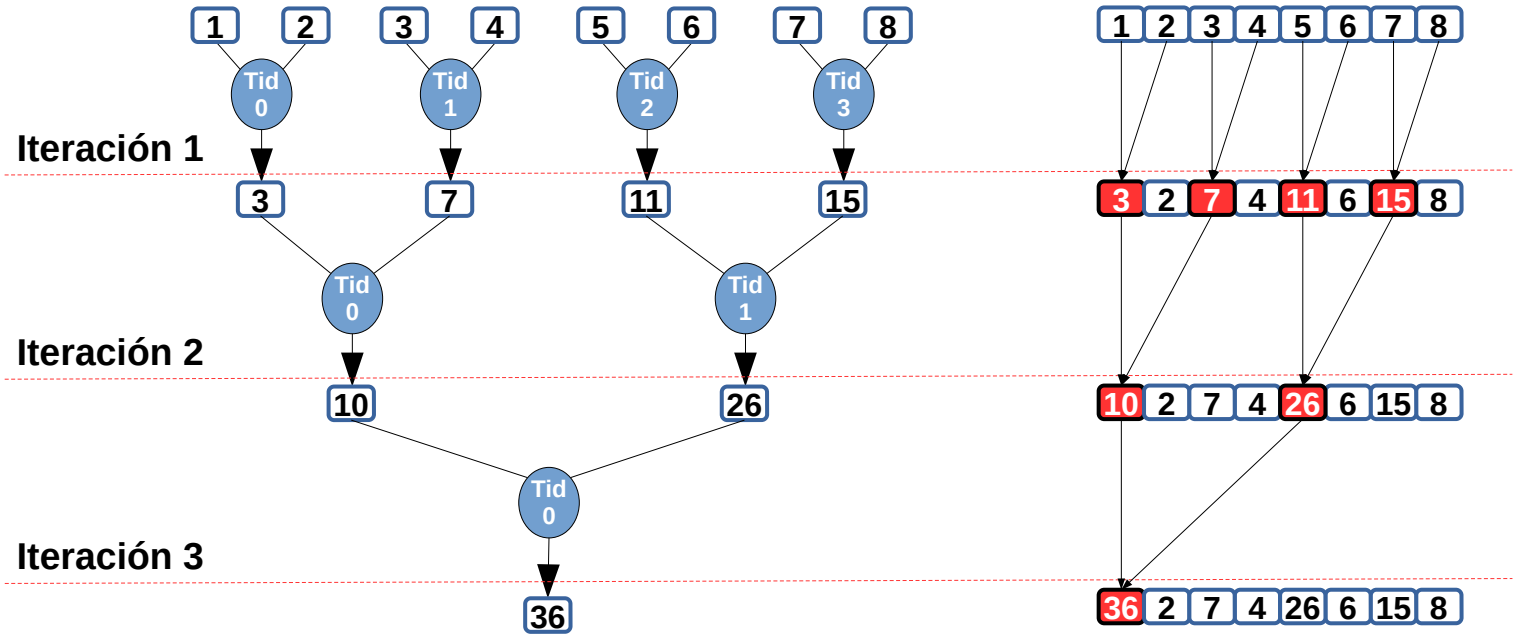
Suma por reducción
Estrategia

- Para la siguiente iteración se invoca nuevamente al kernel con la mitad de hilos de la iteración anterior y se sigue con la misma estrategia.



Suma por reducción
Estrategia

- En la última iteración se invoca al kernel con un sólo hilo, el resultado final queda en la primer posición del vector.



Agenda

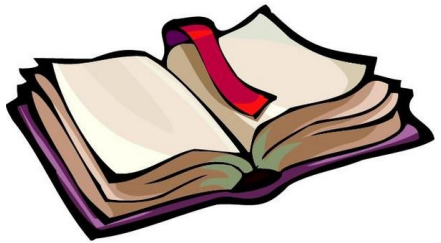
I. Reducción

II. Suma por reducción

   i. Estrategia

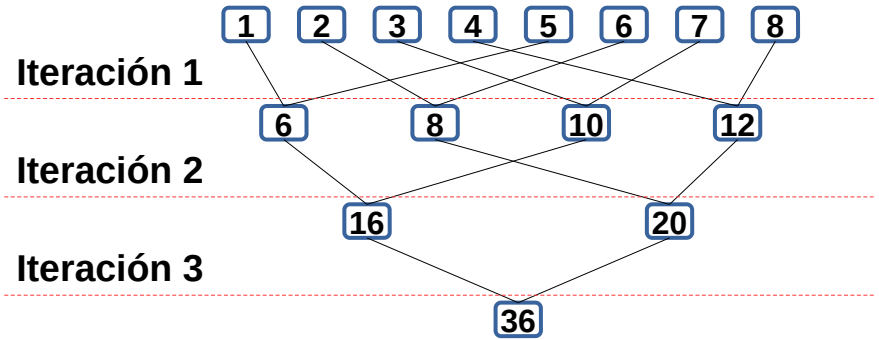
   ii. Alternativa para mejora de acceso a memoria

   iii. Código CUDA



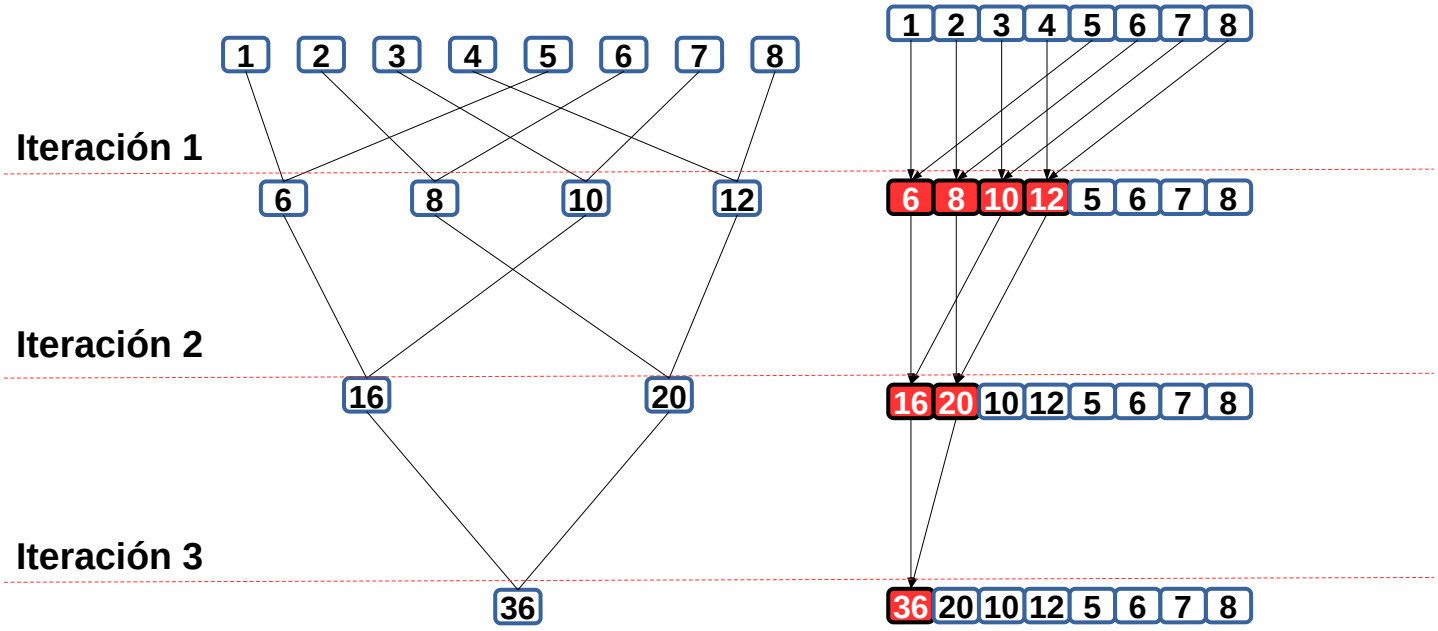
Suma por reducción
Alternativa para mejora de acceso a memoria

- Una alternativa, que mejora el acceso a memoria, es utilizar un patrón de acceso a posiciones contiguas.



Suma por reducción
Alternativa para mejora de acceso a memoria

- La estrategia con el nuevo patrón de acceso:



Agenda

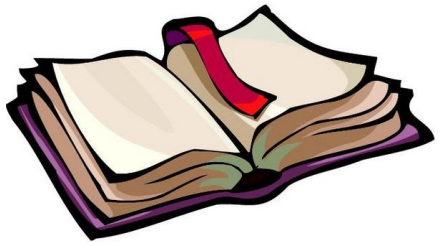
I. Reducción

II. Suma por reducción

   i. Estrategia

   ii. Alternativa para mejora de acceso a memoria

   iii. Código CUDA



Suma por reducción
Código CUDA

- Pseudocódigo:




```
__global__ kernelReduccion("parámetros"){
 int idx = blockDim.x*blockIdx.x + threadIdx.x;
 If (idx < "límite"){
  vglobal[idx] += vglobal[idx + "distancia"];
 }
}
```

```
int main(int argc, char* argv[]){
 int blockSize = 256;
 dim3 dimBlock(blockSize);
…
 for ("nro iteraciones"){
   dim3 dimGrid("en función de N y dimBlock")
   kernelReduccion<<<dimGrid, dimBlock>>>("parámetros");
   cudaDeviceSynchronize();
 }
...
}
```

Suma por reducción
Código CUDA

- Pseudocódigo:

```
__global__ kernelReduccion("parámetros"){
 int idx = blockDim.x*blockIdx.x + threadIdx.x;
 If (idx < "límite"){
  vglobal[idx] += vglobal[idx + "distancia"];
 }
}
```

```
int main(int argc, char* argv[]){
 int blockSize = 256;
 dim3 dimBlock(blockSize);
…
 for ("nro iteraciones"){
   dim3 dimGrid("en función de N y dimBlock")
   kernelReduccion<<<dimGrid, dimBlock>>>("parámetros");
   cudaDeviceSynchronize();
 }
...
}
```



El tamaño del problema puede ser menor a la cantidad de hilos por bloque.
Un hilo podría leer una posición de memoria más allá de los límites del problema.

